

DynamoDB Storage Design

Storing FBAR Filings + PDFs in DynamoDB — Design Note

TL;DR

Store the **JSON filing record** (what's already in `synthetic_fbar_filings.json`) as a DynamoDB item. Store the **PDF bytes** in **S3**, not inline in DynamoDB — even though these particular specimen PDFs are small enough to technically fit, it's the wrong long-term pattern. The DynamoDB item holds an `pdfDocument` pointer block (bucket, key, hash, size) — which is exactly the shape already added to each record in the JSON.

Why not put the PDF bytes directly in the DynamoDB item

- **Hard item size limit.** DynamoDB caps every item (all attributes combined) at **400 KB**. The 50 specimen PDFs here run ~4.2–5.6 KB each, so they'd technically fit Base64-encoded — but a real multi-account, multi-page FBAR (Parts II/III/IV all populated, continuation pages) can grow well past that, and you don't want a storage pattern that silently breaks once a filer has enough foreign accounts.
- **Cost/throughput.** DynamoDB read/write capacity is priced and provisioned around small, frequent item access. Large binary blobs blow up RCU/WCU consumption and crowd out the table's real workload (case queries, status lookups).
- **No native binary serving.** S3 gives you presigned URLs, range requests, versioning, lifecycle policies (e.g., move to Glacier after case closure), and server-side encryption (SSE-KMS) for the document itself — none of which DynamoDB provides for blob content.

This is the standard AWS reference pattern: **metadata in DynamoDB, blob in S3, DynamoDB item carries the S3 pointer.**

DynamoDB table shape

Table: `fbar-filings`

Attribute	Type	Notes
<code>bsaId</code> (PK)	S	Partition key, e.g. 310000000000001
<code>recordType</code> (SK)	S	Sort key, constant "FILING" for the main record (leaves room for a future "AUDIT#<ts>" item type in the same partition)
<code>taxYear</code>	N	GSI partition key candidate
<code>filingStatus</code>	S	GSI partition key candidate
<code>caseId</code>	S	Set once linked to a case; GSI candidate for "filings by case" lookups
<code>filer</code>	M	Nested map, mirrors the JSON <code>filer</code> object
<code>jointFilers</code>	L	List of maps
<code>financialAccounts</code>	L	List of maps
<code>signature</code>	M	Nested map

Attribute	Type	Notes
pdfDocument	M	{fileName, s3Bucket, s3Key, contentType, sizeBytes, sha256, generatedFrom, note}
updatedAt	S	ISO-8601, for optimistic concurrency / change tracking

Suggested Global Secondary Indexes: - GSI-TaxYearStatus: PK taxYear, SK filingStatus — powers GET /filings?taxYear=&filingStatus= - GSI-CaseId: PK caseId — powers “all filings linked to case X”

Example item (low-level DynamoDB JSON, abbreviated)

```
{
  "bsaId": {"S": "31000000000001"},
  "recordType": {"S": "FILING"},
  "taxYear": {"N": "2023"},
  "filingStatus": {"S": "Accepted"},
  "caseId": {"NULL": true},
  "pdfDocument": {
    "M": {
      "fileName": {"S": "FBAR_31000000000001.pdf"},
      "s3Bucket": {"S": "irs-cm-fbar-documents-dev"},
      "s3Key": {"S": "fbar-pdfs/2023/FBAR_31000000000001.pdf"},
      "contentType": {"S": "application/pdf"},
      "sizeBytes": {"N": "4256"},
      "sha256": {"S": "46bd719e4ccd7275392308fefa085597895b39adb97a7b90147080af0c2bed02"}
    }
  }
}
```

S3 layout

s3://irs-cm-fbar-documents-dev/fbar-pdfs/{taxYear}/FBAR_{bsaId}.pdf

- Enable **SSE-KMS** with a customer-managed key scoped to the case management app’s role.
- Block all public access at the bucket level; the case UI fetches via short-lived presigned URLs (or through the API, which proxies/streams after an authz check — see AUTH-04 in fbar_test_scenarios.md for the row-level-authz test case this implies).
- Store the sha256 in the DynamoDB item so the app can verify PDF integrity on read without re-hashing on every request.

Retrieval flow for the case management app

1. GET /api/v1/fbar/filings/{bsaId} → DynamoDB GetItem, returns filing JSON incl. pdfDocument pointer.
2. GET /api/v1/fbar/filings/{bsaId}/pdf → API generates an S3 presigned GET URL (or streams the object) after the same row-level authz check used for the JSON record — never expose the bucket/key directly to the browser without going through authz.
3. Case UI renders/downloads the PDF from the presigned URL.

This keeps the JSON (queryable, filterable, small) and the PDF (large, immutable, served as a file) on the AWS services each is actually designed for, while still giving the case worker one linked view of “the record” and “the document.”